Hotel Management System

Student information & URLs

Full name: Raif El-Khatib

Student ID: GH1033530

GitHub Link: https://github.com/raifkhatib/Hotel-Management-System

Introduction

Managing a hotel is not a simple task. Staff need to track guests, assign rooms, handle bookings, process checkouts, and keep records. All these things need to be done at the same time, doing this manually is slow, and errors could easily be made, which is why software solutions are needed and are so important in the hospitality industry.

In this project, I built a Hotel Management System using Java. The idea was to create a console-based application that simulates the core operations of a real hotel, as in adding guests, managing different room types, making bookings, processing checkouts, and generating bills automatically.

System Architecture

The system follows a modular, object-oriented approach where each class has its own clear and specific job. This makes the code easier to understand, maintain, and improve in the future. The main parts of the system include:

Displayable Interface: The starting point. It defines a single method, Display Info (), that every major class in the system must have. This keeps all objects consistent, and you can always ask any object for it to display its information. Each class does this in its own way, but they all have that ability guaranteed.

Person Abstract Class: The base for all people in the system. It holds shared details such as ID, name, email, and phone number. I made it abstract because in a hotel you are not just a person, you must have a role like a guest member or a staff member. So instead of having a repetition, the shared logic is written once and reused.

Guest and Staff, Subclasses of Person: Both extend Person and inherit its shared details. A Guest has a nationality and earns loyalty points, while a Staff member has a role and a salary. Rather than writing the same basic information twice, I just let both classes build on top of Person, and each one adds only what it needs.

Room, Abstract Class: Represents a hotel room in general. It holds shared details like the room number and whether the room is available or not. I made it abstract for the same reason as a person

because in a real hotel, every room has a specific type. You would never just have a room with no category, so it made no sense to allow that in the code either.

Standard Room, Deluxe Room, Suite and Subclasses of Room: These are the three actual room types in the hotel. Each one extends Room and has its own price per night. This is where inheritance and polymorphism show up together with the same structure but with different behavior.

Booking: This is the class that connects everything. It links a guest to a room, stores the check in and check out dates, and automatically works out the total bill based on how many nights they stayed and what type of room they booked.

Hotel Manager: This is the brain of the whole system. It keeps track of all the guests, rooms, staff, and bookings, and it is the class that handles everything the user does through the menu.

File Handler: This class handles saving and loading data. Every time the program closes, the data is written to a file, and when it opens again, everything is loaded back. Without this, all the information would be lost every time the program stops running.

Implementation

1) Displayable Interface

The first thing I created was the Displayable interface. It has one method, Display Info (), which every major class in the system must have. In Java, an interface works like a contract for any class that implements it is required to provide its own version of that method.

```
java
Public interface Displayable {
   void displayInfo();
}
```

2) **Person Abstract Class**

I created the Person abstract class, which implements the Displayable interface and holds shared details like ID, name, email, and phone number. These are protected so subclasses like Guest and Staff can use them directly. Getters let other parts of the program read these values safely without changing them, which is encapsulation in practice. The class is abstract because you would never create a plain person in a hotel everyone is either a guest or staff member.

```java
abstract class Person implements Displayable {
   protected int id;
   protected String name;
   protected String email;
   protected String phone;

   public Person(int id, String name, String email, String phone) {
      this.id = id;
      this.name = name;
      this.email = email;
      this.phone = phone;
   }

   public int getId()      { return id; }
   public String getName()  { return name; }
   public String getEmail() { return email; }
   public String getPhone() { return phone; }
}
```

3) **Guest Class**

The Guest class extends Person, so it inherits shared attributes like ID, name, email, and phone. It also adds guest-specific details: nationality and a loyalty points counter, which starts at zero and increases with

each booking. The Display Info () method is overridden to show all guest details, fulfilling the interface contract. This class shows inheritance, encapsulation, and method overriding all working together.

```
class Guest extends Person {
   private String nationality;
   private int loyaltyPoints;

   public Guest(int id, String name, String email, String phone, String nationality) {
      super(id, name, email, phone);
      this.nationality = nationality;
      this.loyaltyPoints = 0;
   }

   public String getNationality()   { return nationality; }
   public int getLoyaltyPoints()    { return loyaltyPoints; }

   public void addLoyaltyPoints(int points) {
      this.loyaltyPoints += points;
      System.out.println("  " + points + " loyalty points added to " + name + ". Total: " + loyaltyPoints);
   }
```

```
    @Override
    public void displayInfo() {
        System.out.println("--- Guest Info ---");
        System.out.println("ID        : " + id);
        System.out.println("Name      : " + name);
        System.out.println("Email     : " + email);
        System.out.println("Phone     : " + phone);
        System.out.println("Nationality : " + nationality);
        System.out.println("Loyalty Pts : " + loyaltyPoints);
    }
}
```

4) **Staff Class**

Like Guest, the Staff class extends Person and gets all the shared details automatically. It adds two things just for staff: role and salary. The Display Info () method is given a new version to show the staff members details. This works the same way as Guest, which keeps the code consistent and avoids repeating the same parts. Both classes use the same base but add only what they need.

```
class Staff extends Person {
    private String role;
    private double salary;
```

```
    public Staff(int id, String name, String email, String phone, String role,
double salary) {
        super(id, name, email, phone);
        this.role = role;
        this.salary = salary;
    }

    public String getRole()    { return role; }
    public double getSalary()  { return salary; }

    @Override
    public void displayInfo() {
        System.out.println("--- Staff Info ---");
        System.out.println("ID     : " + id);
        System.out.println("Name   : " + name);
        System.out.println("Email  : " + email);
        System.out.println("Phone  : " + phone);
        System.out.println("Role   : " + role);
        System.out.println("Salary : $" + salary);
    }
}
```

5) **Room Abstract Class**

The Room abstract class is the base for all room types in the system. It has shared details like the room number and whether the room is available. Every room starts as available. I also added an abstract method getPricePerNight (), which means every room type must decide its own price because different rooms cost different amounts. Room also uses the Displayable interface, so every room type must have its own Display Info () method to show its details.

```
abstract Class Room implements Displayable {

   protected int roomNumber;

   protected boolean isAvailable;


   public Room(int roomNumber) {

      this.roomNumber = roomNumber;

      this.isAvailable = true;

   }
```

```
    public int getRoomNumber()     { return roomNumber; }

    public boolean isAvailable()    { return isAvailable; }

    public void setAvailable(boolean status) { this.isAvailable = status; }


    public abstract double getPricePerNight();

}
```

6) **StandardRoom, DeluxeRoom and Suite**

There are three room types in the system. Each one extends Room and has its own version of getPricePerNight () and Display Info (). The Standard Room costs $80 per night, Deluxe Room $150, and Suite $300. This is polymorphism, all three are Room objects, but each one works differently based on its type. The price is not set in the Room class because each type has a different price, so each subclass sets its own.

```
class StandardRoom extends Room { ... }  // $80 per night

class DeluxeRoom extends Room   { ... }  // $150 per night

class Suite extends Room       { ... }  // $300 per night
```

7) **Booking Class**

The Booking class brings everything together. It links a Guest to a Room and stores the check-in and check-out dates. The total bill is calculated automatically by counting the nights between the dates and multiplying the room price per night. I used Javas Local Date and Chrono Unit to make the date calculations easy. Each booking also gets a unique ID that increases automatically, so no two bookings have the same ID.

```
class Booking {

   private static int bookingCounter = 1;

   private int bookingId;

   private Guest guest;

   private Room room;

   private LocalDate checkIn;

   private LocalDate checkOut;

   private double totalBill;


   public Booking(Guest guest, Room room, LocalDate checkIn,
LocalDate checkOut) {
```

```
        this.bookingId = bookingCounter++;
        this.guest = guest;
        this.room = room;
        this.checkIn = checkIn;
        this.checkOut = checkOut;
        long nights = ChronoUnit.DAYS.between(checkIn, checkOut);
        this.totalBill = nights * room.getPricePerNight();
    }
    ...
}
```

8) **Hotel Manager Class**

The Hotel Manager class is the main part of the system. It keeps four lists: guests, rooms, staff, and bookings, and has all the methods the user uses through the menu.

When a booking is made, it finds the guest by ID and the room by number, checks that the room is available, calculates the bill, marks the room as unavailable, and adds loyalty points to the guest. When a guest checks out, the room becomes available again.

All the display methods go through the lists and call Display Info () on each object. This is polymorphism in action, the same method gives different results depending on the type of object.

```
class HotelManager {
   private List<Guest> guests = new ArrayList<>();
   private List<Room> rooms = new ArrayList<>();
   private List<Staff> staff = new ArrayList<>();
   private List<Booking> bookings = new ArrayList<>();

   public void addGuest(Guest guest) { ... }
   public void addRoom(Room room) { ... }
   public void addStaff(Staff member) { ... }
   public void makeBooking(int guestId, int roomNumber, LocalDate
checkIn, LocalDate checkOut) { ... }
   public void checkOut(int bookingId) { ... }
   public void displayAllGuests() { ... }
   public void displayAllRooms() { ... }
   public void displayAllBookings() { ... }
   public void displayAllStaff() { ... }
```

}

9) **File Handler Class**

The File Handler class takes care of saving and loading data to text files. It has three static methods: save Guests (), load Guests () and Save Bookings ().

When guests are saved, all their details are written in one line with commas between the values. When the program loads, it reads each line and recreates the Guest objects from the saved data. This way, information is not lost when the program closes, making the system feel more like a real application. I used try-catch blocks to handle file errors safely, so the program will not crash.

```
class FileHandler {
   private static final String GUESTS_FILE = "guests.txt";
   private static final String BOOKINGS_FILE = "bookings.txt";

   public static void saveGuests(List<Guest> guests) { ... }
   public static List<Guest> loadGuests() { ... }
   public static void saveBookings(List<Booking> bookings) { ... }
}
```

10) **Main Class and Menu**

The Main class is where everything comes together. When the program starts, it automatically adds 5 rooms of different types and 2 staff members to the system. The user is then presented with a menu that keeps running in a loop until they choose to exit. Depending on the choice, the program calls the right method in Hotel Manager. The Add Guest (), make Booking (), and checkout () methods handle user input using a Scanner and pass the data to the manager. The menu is built using a do-while loop and a switch statement, which keeps the code clean and easy to follow.

```
class Main {
  static HotelManager manager = new HotelManager();
  static java.util.Scanner scanner = new java.util.Scanner(System.in);

  public static void main(String[] args) {
    // default rooms and staff added on startup
    // menu runs in a do-while loop
    // switch handles each user choice
  }

  static void addGuest() { ... }
  static void makeBooking() { ... }
  static void checkOut() { ... }
}
```

Results

1) Adding a guest:

```
Run    Main
0. Exit
Your choice: 1
Enter guest ID: 1
Enter name: hadi rhayel
Enter email: hadi@gmail.com
Enter phone: 123456789
Enter nationality: lebanese
Guest hadi rhayel added successfully!

============================
```

2) Displaying all rooms

```
Run    Main
0. Exit
Your choice: 5
--- Room Info ---
Room Number : 101
Type        : Standard Room
Price/Night : $80.0
Available   : true

--- Room Info ---
Room Number : 102
```

```
Room Number : 102
Type        : Standard Room
Price/Night : $80.0
Available   : true

--- Room Info ---
Room Number : 201
Type        : Deluxe Room
Price/Night : $150.0
Available   : true
```

```
Room Number : 202
Type        : Deluxe Room
Price/Night : $150.0
Available   : true

--- Room Info ---
Room Number : 301
Type        : Suite
Price/Night : $300.0
Available   : true
```

3) Making booking

```
Run    Main
0. Exit
Your choice: 2
Enter guest ID: 1
Enter room number: 101
Enter check-in date (YYYY-MM-DD): 2025-01-01
Enter check-out date (YYYY-MM-DD): 2025-01-05
  10 loyalty points added to hadi rhayel. Total: 10
Booking confirmed! Bill: $320.0

==============================
```

4) Check out

```
Run    Main
5. Display All Rooms
6. Display All Bookings
7. Display All Staff
8. Save Data
0. Exit
Your choice: 3
Enter booking ID: 1
Guest hadi rhayel checked out. Total paid: $320.0

==============================
```

5) Display all guests

```
Run    Main
8. Save Data
0. Exit
Your choice: 4
--- Guest Info ---
ID          : 1
Name        : hadi rhayel
Email       : hadi@gmail.com
Phone       : 123456789
Nationality : lebanese
Loyalty Pts : 10
```

6) Display all bookings

```
Run    Main
8. Save Data
0. Exit
Your choice: 6
--- Booking Info ---
Booking ID  : 1
Guest       : hadi rhayel
Room        : 101
Check In    : 2025-01-01
Check Out   : 2025-01-05
Total Bill  : $320.0
```

7) Display all staff

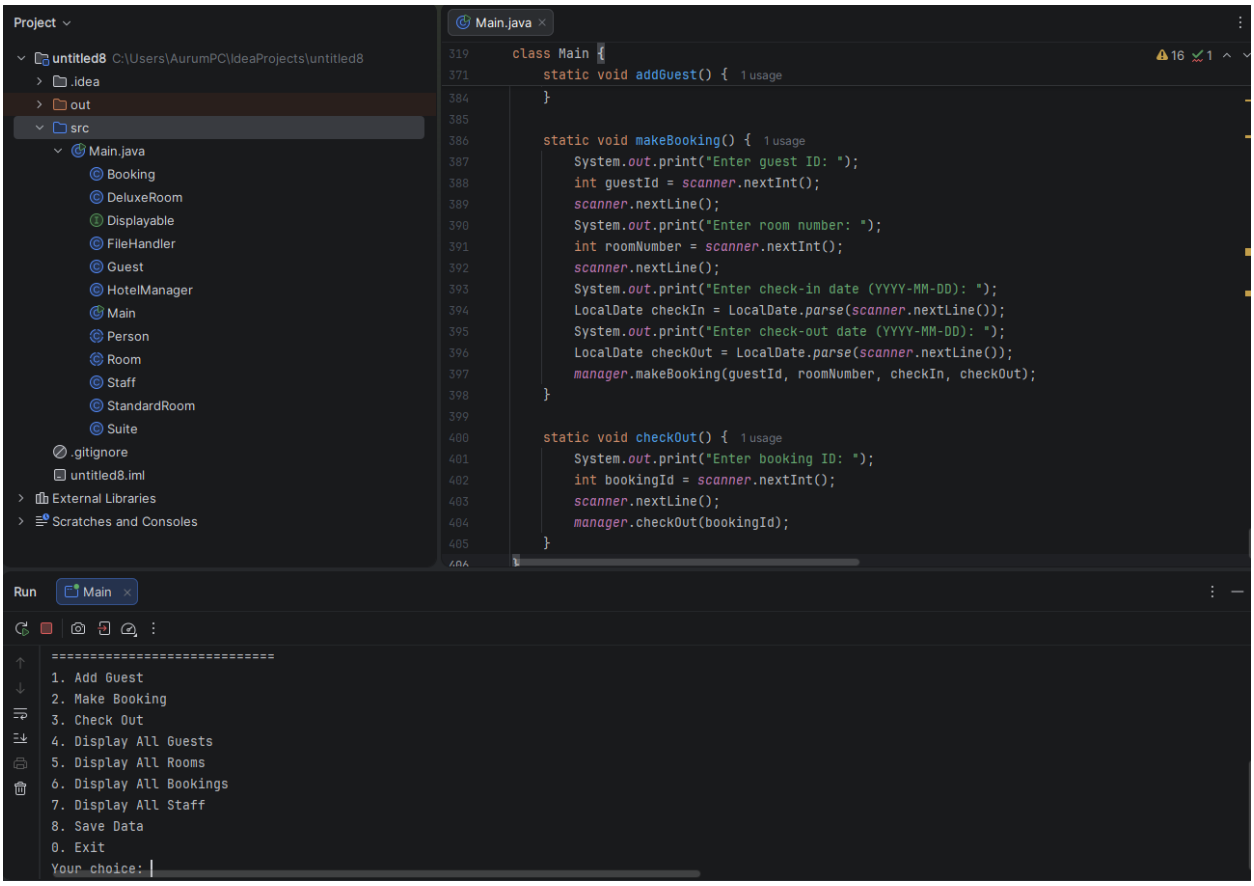
```
Run    Main
Your choice: 7
--- Staff Info ---
ID     : 1
Name   : Alice
Email  : alice@hotel.com
Phone  : 123456
Role   : Manager
Salary : $5000.0

--- Staff Info ---
```

```
Run    Main
--- Staff Info ---
ID     : 2
Name   : Bob
Email  : bob@hotel.com
Phone  : 789012
Role   : Receptionist
Salary : $3000.0


----------------------------
```

And finally, this shows that the code runs perfectly.



Challenges and Solutions

One of the first challenges I ran into was understanding how Java handles public classes and file names. Java requires that every public class or interface be in its own file with the exact same name. When the first code was written, I put everything in one file which caused errors. I solved this by restructuring everything into a single file and removing the public keyword from classes that did not need it.

Date Handling Calculating the number of nights between two dates was something I did not do before to be honest. I solved this by using Javas Local Date class along with Chrono Unit.DAYS.between () which made the calculation clean and accurate without doing any manual math.

I had to make sure a room could not be booked twice. I solved this by adding a Boolean is an Available flag to the Room class that gets set to false when a room is booked and back to true when the guest checks out.

Connecting objects together, the Booking class needed to reference both a Guest and a Room object at the same time. Making sure the right objects were being passed around the system correctly took some planning, but using Array Lists and searching by ID solved this easily.

Conclusion and Future Work

In this project, I built a working Hotel Management System in Java that does the main tasks of a real hotel. The system lets users add guests, manage room types, make bookings, handle checkouts, and save data to files. All parts were tested and they all worked well.

In this project, I used Object-Oriented Programming in a simple way. I used interfaces, abstract classes, inheritance, encapsulation, and polymorphism. I used them because since helped the system work better, not just for theory.

For future improvements, there are some things I could add such as:

I can create a simple graphical user interface, so users do not have to use the console. I can also use a database like MySQL instead of text files to store data more safely. Also, it would be helpful to add options to update or delete guests and book details, and to find guests and booking names quickly, a search feature will be helpful. I can also add a report feature to show monthly revenue and room usage. Finally, I can connect the system to online booking using an API.

Overall, this project was a good learning experience. It helped me understand how to design and build a real Java application from the beginning. I also got a clearer idea of how object-oriented systems work in real life.